



3. Test.exe

문제 파일 Test.exe

test.zip

```

Enter plaintext (ASCII string): tset
Output: 08 56 12 24 1D 68 05 29 14 57 8A BD B4 A5 CB 75

Enter plaintext (ASCII string): test
Output: C2 77 39 E6 D1 36 B2 03 BB B2 06 D2 56 3E 8A 78

Enter plaintext (ASCII string):

```

문자열을 입력하면 의문의 Hex값을 확인

문자열을 입력받은 후 Hex 값으로 변환하여 출력해주는 것 같음

IDA로 확인

```

17| while ( 1 )
18| {
19|     Dest.m128i_i64[0] = 10374466i64;
20|     *(__int16 *)((char *)&Dest.m128i_i16[7] + 1) = 0;
21|     v12 = 0;
22|     Dest.m128i_i32[2] = 7733248;
23|     Dest.m128i_i16[6] = 0;
24|     Dest.m128i_i8[14] = 40;
25|     Src = -634829520;
26|     v9 = 1630871125;
27|     v10 = 1142168362;
28|     v11 = 326978798;
29|     memset(&Dst, 0, 0xA0ui64);
30|     memset(&v14, 0, 0xB0ui64);
31|     v13.m128i_i64[0] = -5771155837093558958i64;
32|     v13.m128i_i64[1] = 46703543839772i64;
33|     sub_1400012A0("Enter plaintext (ASCII string): ");
34|     v3 = (FILE *)_acrt_iob_func(0i64);
35|     fgets(&Buf, 17, v3);
36|     v7 = 0;
37|     Dest = 0i64;
38|     strncpy(Dest.m128i_i8, &Buf, 0x10ui64);
39|     _mm_storeu_si128(&Dest, _mm_xor_si128(_mm_loadu_si128(&Dest), _mm_loadu_si128(&v13)));
40|     sub_140001000(&Src);
41|     sub_140001470(&Dest, &Dst, &v14);
42|     sub_1400012A0("Output: ");
43|     for ( i = 0i64; i < 16; ++i )
44|         sub_1400012A0("%02X ");
45|     puts("\n\n");
46| }
47| }

```

fgets(&Buf, 17, v3); //fgets 함수로 값을 입력 받음

v7 = 0;

Dest = 0i64

strncpy(Dest.m128i_i8, &Buf, 0x10ui64); // 입력받은 값을 구조체에 복사

_mm_storeu_si128(&Dest, _mm_xor_si128(_mm_loadu_si128(&Dest), _mm_loadu_si128(&v13)));

////?? 이 부분은 뭘까요(피..피카츄)?

sub_140001000(&Src);

sub_140001470(&Dest, &Dst, &v14);

sub_1400012A0("Output: "); // printf

for (i = 0i64; i < 16; ++i)

```
sub_1400012A0("%02X");
puts("\n\n");
```

1. fgets 함수로 값을 입력을 받음
2. 입력값을 구조체에 복사
3. 그리고 뭔가를 수행해서 출력???

m128i (__mm_storeu_si128?, mm_xor_si128?, __mm_loadu_si128?)

This is SIMD!! (Single Instruction Multiple Data)

하나의 명령어로 다중(병렬) 데이터를 처리하는 것을 말하는데, 벡터(vector)연산이라고도 함

여기서 말하는 다중데이터는 일반적인 크기의 데이터가 아니라 64비트 128비트 또는 256비트 크기를 가진 자료를 의미 → 교수님이 말씀하신 128비트의 의미

→ 참고만 합시다

<emmintrin.h> - SSE2					
typedef	union	__declspec(intrin_type)	_CRT_ALIGN(16)	__m128i	{
		__int8	m128i_i8[16];		
		__int16	m128i_i16[8];		
		__int32	m128i_i32[4];		
		__int64	m128i_i64[2];		
	unsigned	__int8	m128i_u8[16];		
	unsigned	__int16	m128i_u16[8];		
	unsigned	__int32	m128i_u32[4];		
	unsigned	__int64	m128i_u64[2];		
				__m128i;	}

<xmmmintrin.h> - SSE					
typedef	union	__declspec(intrin_type)	_CRT_ALIGN(16)	__m128	{
		float	m128_f32[4];		
	unsigned	__int64	m128_u64[2];		
		__int8	m128_i8[16];		
		__int16	m128_i16[8];		
		__int32	m128_i32[4];		
		__int64	m128_i64[2];		
	unsigned	__int8	m128_u8[16];		
	unsigned	__int16	m128_u16[8];		
	unsigned	__int32	m128_u32[4];		
				__m128;	}

```
_mm_storeu_si128((__m128i *)&Dest, _mm_xor_si128(_mm_loadu_si128(&Dest), _mm_loadu_si128(&v13)));
sub_140001000(&Src, (__int64)&Dst, &v14);
```

해당 주소를 확인해보면... sub_140001000

```
memmove(a3, Src, 0x10ui64);
v6 = 0i64; // → v6 함수는 64비트 자료형이면서 현재 값은 0
v7 = (__int64)(v3 + 17);
do
{
    v8 = *(unsigned __int8 *)(v7 - 4);
    v9 = *(unsigned __int8 *)(v7 - 3);
```


Keypoint

- 라운드 과정이니까 AES 암호화 수행 후 들어가 있는 값을 확인하면?

[illegible]

- 주소 확인해보면 140003380

```

00000000140003380 63 7C 77 7B F2 6B 6F C5 30 01 67 2B FE D7 AB 76
00000000140003390 CA 82 C9 7D FA 59 47 F0 AD D4 A2 AF 9C A4 72 C0
000000001400033A0 B7 FD 93 26 36 3F F7 CC 34 A5 E5 F1 71 D8 31 15
000000001400033B0 04 C7 23 C3 18 96 05 9A 07 12 80 E2 EB 27 B2 75
000000001400033C0 09 83 2C 1A 1B 6E 5A A0 52 3B D6 B3 29 E3 2F 84
000000001400033D0 53 D1 00 ED 20 FC B1 5B 64 CB BE 39 4A 4C 58 CF
000000001400033E0 D0 EF AA FB 43 4D 33 85 45 F9 02 7F 50 3C 9F A8
000000001400033F0 51 A3 40 8F 92 9D 38 F5 BC B6 DA 21 10 FF F3 D2
00000000140003400 CD 0C 13 EC 5F 97 44 17 C4 A7 7E 3D 64 5D 19 73
00000000140003410 60 81 4F DC 22 2A 90 88 46 EE B8 14 DE 5E 0B DB
00000000140003420 E0 32 3A 0A 49 06 24 5C C2 D3 AC 62 91 95 E4 79
00000000140003430 E7 C8 37 6D 8D D5 4E A9 6C 56 F4 EA 65 7A AE 08
00000000140003440 BA 78 25 2E 1C A6 B4 C6 E8 DD 74 1F 4B BD 8B 8A
00000000140003450 70 3E B5 66 48 03 F6 0E 61 35 57 B9 86 C1 1D 9E
00000000140003460 E1 F8 98 11 69 D9 8E 94 9B 1E 87 E9 CE 55 28 DF
00000000140003470 8C A1 89 0D BF E6 42 68 41 99 2D 0F B0 54 BB 16

```

```
.rdata:00000000140003380 byte_140003380 db 63h, 7Ch, 77h, 7Bh, 0F2h, 6Bh, 6Fh, 0C5h, 30h, 1, 67h
.rdata:00000000140003380 db 2Bh, 0FEh, 0D7h, 0ABh, 76h, 0CAh, 82h, 0C9h, 7Dh, 0FAh
.rdata:00000000140003380 db 59h, 47h, 0F0h, 0ADh, 0D4h, 0A2h, 0AFh, 9Ch, 0A4h, 72h
.rdata:00000000140003380 db 0C0h, 0B7h, 0FDh, 93h, 26h, 36h, 3Fh, 0F7h, 0CCh, 34h
.rdata:00000000140003380 db 0A5h, 0E5h, 0F1h, 71h, 0D8h, 31h, 15h, 4, 0C7h, 23h
.rdata:00000000140003380 db 0C3h, 18h, 96h, 5, 9Ah, 7, 12h, 80h, 0E2h, 0EBh, 27h
.rdata:00000000140003380 db 0B2h, 75h, 9, 83h, 2Ch, 1Ah, 1Bh, 6Eh, 5Ah, 0A0h, 52h
.rdata:00000000140003380 db 3Bh, 0D6h, 0B3h, 29h, 0E3h, 2Fh, 84h, 53h, 0D1h, 0
.rdata:00000000140003380 db 0EDh, 20h, 0FCh, 0B1h, 5Bh, 6Ah, 0C8h, 0BEh, 39h, 4Ah
```

.rdata:0000000140003380	db 4Ch, 58h, 0CFh, 0D0h, 0EFh, 0AAh, 0FBh, 43h, 4Dh, 33h
.rdata:0000000140003380	db 85h, 45h, 0F9h, 2, 7Fh, 50h, 3Ch, 9Fh, 0A8h, 51h, 0A3h
.rdata:0000000140003380	db 40h, 8Fh, 92h, 9Dh, 38h, 0F5h, 0BCh, 0B6h, 0DAh, 21h
.rdata:0000000140003380	db 10h, 0FFh, 0F3h, 0D2h, 0CDh, 0Ch, 13h, 0ECh, 5Fh, 97h
.rdata:0000000140003380	db 44h, 17h, 0C4h, 0A7h, 7Eh, 3Dh, 64h, 5Dh, 19h, 73h
.rdata:0000000140003380	db 60h, 81h, 4Fh, 0DCh, 22h, 2Ah, 90h, 88h, 46h, 0EEh
.rdata:0000000140003380	db 0B8h, 14h, 0DEh, 5Eh, 0Bh, 0DBh, 0E0h, 32h, 3Ah, 0Ah
.rdata:0000000140003380	db 49h, 6, 24h, 5Ch, 0C2h, 0D3h, 0ACh, 62h, 91h, 95h, 0E4h
.rdata:0000000140003380	db 79h, 0E7h, 0C8h, 37h, 6Dh, 8Dh, 0D5h, 4Eh, 0A9h, 6Ch
.rdata:0000000140003380	db 56h, 0F4h, 0EAh, 65h, 7Ah, 0AEh, 8, 0BAh, 78h, 25h
.rdata:0000000140003380	db 2Eh, 1Ch, 0A6h, 0B4h, 0C6h, 0E8h, 0DDh, 74h, 1Fh, 4Bh
.rdata:0000000140003380	db 0BDh, 8Bh, 8Ah, 70h, 3Eh, 0B5h, 66h, 48h, 3, 0F6h, 0Eh
.rdata:0000000140003380	db 61h, 35h, 57h, 0B9h, 86h, 0C1h, 1Dh, 9Eh, 0E1h, 0F8h
.rdata:0000000140003380	db 98h, 11h, 69h, 0D9h, 8Eh, 94h, 9Bh, 1Eh, 87h, 0E9h
.rdata:0000000140003380	db 0CEh, 55h, 28h, 0DFh, 8Ch, 0A1h, 89h, 0Dh, 0BFh, 0E6h
.rdata:0000000140003380	db 42h, 68h, 41h, 99h, 2Dh, 0Fh, 0B0h, 54h, 0BBh, 16h\

AES의 실제 S-Box 값

```

81  0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x81, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
82  0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
83  0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
84  0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
85  0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
86  0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
87  0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
88  0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
89  0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
90  0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
91  0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
92  0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
93  0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
94  0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
95  0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
96  0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16 };

```

⇒ 결국 문자열을 입력하면 문자열을 아스키 값으로 입력받은 뒤 AES 암호화를 진행하고 출력해줌

Reference

<https://blog.naver.com/fs0608/221650925743>

<https://openmynotepad.tistory.com/97>

<https://pi-log.tistory.com/61>

[AES.pdf](#)

[DES.pdf](#)

<https://github.com/kokke/tiny-AES-c/blob/master/aes.c>